

ENIAD TP4

POO en C++: Héritage, polymorphisme et abstraction

Exercice 1 : Gestion d'une bibliothèque

On désire de créer un application de gestion de bibliothèque avec les spécifications suivants :
Classe de base Article avec les caractéristiques : titre, auteur, annee, disponible qui indique si l'article est disponible pour l'emprunt.

Ajouter le comportement suivant :

Construteur d'initialisation

- void emprunter() : marque l'article comme emprunté
- void retourner() : marque l'article comme retourné
- void afficherDetails() : affiche les détails de l'article

Classe dérivée Livre avec attribut supplémentaires : genre

Classe dérivée Magazine avec attribut supplémentaires : mois le mois de publication du magazine.

Tester votre application.

Exercice 2 : Simulation de ZOO

Nous voulons simuler un ZOO d'animal avec les specifications suivants :

Classe de base Animal avec les attributs : nom, age, sexe et enBonneSante indique si l'animal est en bonne santé

Méthodes :

- Animal(string nom, int age, string sexe) : constructeur pour initialiser les attributs
- void nourrir() : nourrit l'animal (simple message)
- void soigner() : soigne l'animal (simple message)
- void afficherDetails() : affiche les détails de l'animal

Classe dérivée Mammifere :

Attributs supplémentaires :

nombreDePattes : le nombre de pattes du mammifère

Classe dérivée Oiseau :

Attributs supplémentaires :

envergure : (float) l'envergure de l'oiseau en mètres

Classe dérivée Reptile :

Attributs supplémentaires :

venimeux : (bool) indique si le reptile est venimeux ou non

Tester votre application.

Il faut sepparer le prototype et l'implémentation (.h et .cpp)

Exercice 3 : System de gestion des employés

Nous désirons créer une application de gestion des employés avec le cahier de charge suivant :

Classe de base Employe, avec les caractéristiques : nom, salaire, nombreHeures, le nombre d'heures travaillées par semaine et le tauxHoraire.

Méthodes : calculerSalaire() : méthode virtuelle pure pour calculer le salaire de l'employé, à implémenter dans les classes dérivées.

Classe dérivée EmployeTempsPlein :

calculerSalaire() : implémente le calcul du salaire mensuel pour un employé à temps plein (salaire mensuel fixe)

Classe dérivée EmployeTempsPartiel :

calculerSalaire() : implémente le calcul du salaire mensuel pour un employé à temps partiel (salaire horaire * nombre d'heures travaillées par semaine * 4).

Tester votre application en creant un tableau de 5 employés.

Exercice 4 : Gestionnaire de locations de films

L'objectif de cette exercice est de simuler la création d'une application de gestion de locations de films.

Classe mère : Film.

Attributs :

- identifiant : identifiant unique du film (entier)
- titre : le titre du film (chaîne de caractères)
- realisateur : le nom du réalisateur du film (chaîne de caractères)
- duree : la durée du film en minutes (entier)
- prixLocationJour : le prix de location par jour du film (nombre à virgule flottante)

Actions :

- Accesseurs et mutateurs pour chaque attribut
- calculerLocation() : méthode virtuelle pure pour calculer le coût de la location, à implémenter dans les classes dérivées

Classes dérivées (FilmAction, FilmComedie, FilmDrame) avec une caractéristique supplémentaire 'nombre d'explosions' just pour un film d'action.

Méthodes :

- calculerLocation() : implémente le calcul du coût de la location spécifique à chaque genre de film

Gestionnaire de Locations :

Attributs :

- Liste des films disponibles
- Liste des films loués
- Revenus de l'entreprise

Comportement :

louerFilm(film, jours) : enregistre la location d'un film pour un nombre de jours donné

afficherDetailsLocations() : affiche les détails des films loués, y compris le coût total
afficherRevenus() : affiche les revenus totaux de l'entreprise

Interface Utilisateur (test de l'application) :

Menu principal :

- Louer un film
- Afficher les détails des films loués
- Afficher les revenus de l'entreprise
- Quitter